TOPORIA · TOPOLOGY OPTIMISATION

The library: what is implemented, and what is still to add

Every part you can pick in Toporia today, where it comes from and how it came in, followed by the methods still to implement, ordered from least to most effort.

57	11	30
parts in the library	benchmark problems	items still to add

Branch `integration/all` · generated 24 October 2026 from `toporia/plugins/LIBRARY.md`

This page lists **every part you can pick in Toporia today**, what it is, where it comes from and how it got here. The second half is the list of what is still to be added.

- Each kind of part has its own folder and README with the details and the formulas.
- The survey lists every open topology-optimisation code that was found, with links and licences.
- The to-do list is the working plan behind the list below.

Every part on this page:

- appears in the desktop app;
- has a **?** button in the app that shows the same reference as here;
- passes `toporia check <kind>:<name>`.

A test (`tests/apps/test_gui.py`) fails if a part ever stops being reachable from the app.

How a part came in

Mark	Meaning
Port	Translated into Python from a published code (usually MATLAB). The original is named.
Paper	Written here from the paper. No code was copied.
Wraps	Calls an outside Python package. That package is installed separately. Parts whose package is missing are greyed out in the app, with the install command.
Own	Toporia's own idea or standard practice.

In the library today: 57 parts and 11 benchmark problems

Design representations: what the design variables are (folder)

In the app	Name	File	Came in	From
Element densities	`element_density`	element_density.py	Own	Bendsøe 1989; Sigmund's 99-line code
Moving morphable components (constant or variable thickness)	`mmc`	mmc.py	Paper	Guo, Zhang & Zhong 2014; variable thickness as in MMC188 (Zhang et al., *SMO* 2016)
B-spline field	`bspline`	bspline.py	Paper	Qian, *CMAME* 2013

Filters: design to physical density, and fabrication rules (folder)

In the app	Name	File	Came in	From
Density filter	`density`	density.py	Port	top88 (Andreassen et al. 2011); Bruns & Tortorelli 2001; Bourdin 2001
PDE (Helmholtz) filter	`pde`	pde.py	Port	the PDE variant of top88 (top82.m); Lazarov & Sigmund 2011
Sensitivity filter	`sensitivity`	sensitivity.py	Port	Sigmund 1997, as in the 99-line code and top88
Heaviside projection (fixed or volume-preserving threshold)	`heaviside`	heaviside.py	Paper	Guest et al. 2004; Wang, Lazarov & Sigmund 2011; volume-preserving as in top99neo (Ferrari & Sigmund 2020)
Morphology filter (erode / dilate / open / close)	`morphology`	morphology.py	Paper	Sigmund, *SMO* 2007
AM overhang filter	`am`	am.py	Port	Langelaar's AMfilter.m (*SMO* 2017)
Routing radius filter	`routing`	routing.py	Own	inspired by Sigmund 2007
Symmetry filter	`symmetry`	symmetry.py	Own	standard practice
Extrusion (constant cross-section)	`extrusion`	extrusion.py	Paper	Ishii & Aomura 2004
Pattern repetition	`pattern`	pattern.py	Paper	Bendsøe & Sigmund 2003; Almeida, Paulino & Silva 2009
Casting (no undercuts)	`casting`	casting.py	Paper	Gersborg & Andreasen, *SMO* 2011

Material laws: density to stiffness (folder)

In the app	Name	File	Came in	From
SIMP (power law)	`simp`	simp.py	Paper	Bendsøe 1989
RAMP (rational)	`ramp`	ramp.py	Paper	Stolpe & Svanberg 2001
Ordered SIMP (multi-material)	`ordered_simp`	ordered_simp.py	Paper	Zuo & Saitou, *SMO* 2017

Physics models: what computes displacements and gradients (folder, elements in physics/)

In the app	Name	File	Came in	From
2-D Q4 plane stress (Toporia)	`q4`	q4.py, q4_plane_stress.py	Port	the physics half of top88
3-D H8 bricks (Toporia)	`h8`	h8.py, h8_solid.py	Port	top3d (Liu & Tovar, *SMO* 2014)
Linear elasticity, pyMOTO network	`pymoto_elastic`	pymoto_elastic.py	Wraps	pyMOTO (MIT), `pip install pymoto`

Objectives and constraints (folder)

In the app	Name	File	Came in	From
Compliance	`compliance`	compliance.py	Own	the classic objective (Bendsøe & Sigmund 2003)
Material volume	`volume`	volume.py	Own	as in Le et al. 2010 and pyMOTO's stress example
Peak von Mises stress	`stress`	stress.py	Paper	Le, Norato, Bruns, Ha & Tortorelli, *SMO* 2010

Optimisers: how the design moves (folder)

In the app	Name	File	Came in	From
Optimality criteria	`oc`	oc.py	Port	the 99-line code and top88
Optimality criteria, several constraints	`oc_multi`	oc_multi.py	Own	after Zhou & Rozvany 1991
Method of moving asymptotes	`mma`	mma.py	Port	Svanberg 1987, one-constraint form of top88's MMA variant
CONLIN (convex linearisation)	`conlin`	conlin.py	Paper	Fleury & Braibant 1986; Fleury 1989
Mirror descent (SiMPL)	`simpl`	simpl.py	Paper	Keith, Kim, Lazarov & Surowiec, *SMO* 2025
BESO (soft kill)	`beso`	beso.py	Paper	Huang & Xie 2007, 2010
BESO (hard kill)	`beso_hard`	beso_hard.py	Paper	Huang & Xie 2010
TOBS (integer LP)	`tobs`	tobs.py	Paper	Sivapuram & Picelli 2018; the 101-line code of Picelli et al. 2021; solved with HiGHS (SciPy)
Adam (augmented Lagrangian)	`adam`	adam.py	Paper	Kingma & Ba 2015; the volume penalty of TOuNN (Chandrasekhar & Suresh 2021)
Proportional topology optimisation	`pto`	pto.py	Paper	Biyikli & To, *PLoS ONE* 2015
MMA (pyMOTO), GCMMA (pyMOTO)	`pymoto_mma`, `pymoto_gcmma`	pymoto_optimizers.py	Wraps	pyMOTO (MIT)
MMA (mmapy), GCMMA (mmapy)	`mmapy_mma`, `mmapy_gcmma`	mmapy_optimizers.py	Wraps	mmapy, Svanberg's reference code in Python (**GPL-3**, optional), `pip install mmapy`

In the app	Name	File	Came in	From
SLSQP (SciPy)	`scipy_slsqp`	scipy_slsqp.py	Wraps	SciPy (BSD), Kraft 1988
trust-constr (SciPy)	`scipy_trust_constr`	scipy_minimize.py	Wraps	SciPy (BSD)
MMA, CCSAQ, SLSQP, augmented Lagrangian + L-BFGS (NLopt)	`nlopt_mma`, `nlopt_ccsaq`, `nlopt_slsqp`, `nlopt_auglag`	nlopt_optimizers.py	Wraps	NLopt (LGPL/MIT), `pip install nlopt`
Null-space optimiser	`nullspace`	nullspace.py	Wraps	null-space optimizer of Feppon, Allaire & Dapogny 2020 (**GPL-3**, optional), `pip install nullspace_optimizer`

Whole methods: their own loop (folder)

In the app	Name	File	Came in	From
RBF level set	`levelset`	levelset_rbf.py	Port	TOPRBF.m (Luo et al.); Wang & Wang, *CMAME* 2006

Every model can also be paired with every optimiser as a method called `<model>+<optimiser>`, for example `q4+nlopt_mma`. Those pairings are built on demand, so they are not listed here.

Continuation: parameters that change during a run (folder)

In the app	Name	File	Came in	From
Equal steps	`steps`	steps.py	Own	the classic SIMP continuation (Sigmund & Petersson 1998)
Multiply by a factor	`geometric`	geometric.py	Own	the usual β continuation (Guest et al. 2004; Wang et al. 2011)
Linear ramp	`linear`	linear.py	Own	—
When the design settles	`settled`	settled.py	Own	common practice in the projection literature (Wang et al. 2011)

The robust formulation (one design evaluated in several versions) is a solver setting rather than a schedule. See the schedules README and the preset *MBB Beam (Robust)*.

Post-processing: what is made of the finished design (folder)

In the app	Name	File	Came in	From
Threshold to black and white	`threshold`	threshold.py	Own	as method comparisons do (Rojas-Labanda & Stolpe 2015)
Connectivity check	`connectivity`	connectivity.py	Own	—
Feature size	`feature_size`	feature_size.py	Own	measures what Sigmund 2007 and Wang et al. 2011 promise
Grey level (M_nd) over the run	`grey_level`	grey_level.py	Paper	Sigmund 2007
Export outline (SVG), (DXF)	`export_svg`, `export_dxf`	export_svg.py, export_dxf.py	Own	—

In the app	Name	File	Came in	From
Smooth outline (SVG + DXF)	`smooth_outline`	smooth_outline.py	Paper	Subedi et al. 2020 (review); smoothing splines or Chaikin 1974
Export solid (STL)	`export_stl`	export_stl.py	Own	—

Benchmark problems (folder)

In the app	File	From
MBB Beam	mbb_beam.py	the 99-line code and top88
MBB Beam (Robust)	mbb_beam_robust.py	Wang, Lazarov & Sigmund 2011
Cantilever	cantilever.py	the 99-line code
Cantilever 3D	cantilever_3d.py	top3d (Liu & Tovar 2014)
Michell Cantilever	michell_cantilever.py	Michell 1904
L-bracket	l_bracket.py	Duysinx & Bendsøe 1998; Le et al. 2010 (dimensions still to be checked against the papers)
Three-Point Bending	three_point_bending.py	the materials test
Standard Bar	standard_bar.py	Toporia's own
Drone Arm, Drone Arm (Camera View), Drone Arm (Point Loads)	drone_arm.py, drone_arm_camera_view.py, drone_arm_point_loads.py	Toporia's own application case

To add next

Listed from least to most effort. "Code" means open code exists (port it, or check against it). "Paper" means it would be written from the paper. Details and status are in TODO.md.

Small: one file in an existing slot

What	Kind	Source	Notes
Anisotropic PDE filter	filter	Paper: Lazarov & Sigmund 2011	a small extension of `pde`
Moving morphable voids (MMV)	representation	Paper: Zhang et al., *CMAME* 2017	holes instead of bars
Floating projection	optimiser	Paper: Huang, *CMAME* 2021	binary designs without β continuation
Simulated annealing	optimiser	Code: topASA	a "why it loses" baseline
Compliant mechanism (output displacement) and the force-inverter benchmark	objective + problem	Paper: Sigmund 1997	needs springs at input and output
pyMOTO's thermal, dynamic and mechanism modules	models	Code: pyMOTO	already installed: the cheapest new physics
Reference results from the original MATLAB codes (top88, top99neo, top3d, TOBS, MMC, the level-set codes)	tests	Code: run once in GNU Octave or MATLAB	proves the Python versions give the same answers

Medium: new physics or a whole method

What	Source
Heat conduction and thermal compliance	Paper: Bendsøe & Sigmund 2003; Gersborg-Hansen et al. 2006
Eigenfrequency, then linearised buckling	Code: TopBuck250 (Ferrari, Sigmund & Guest 2021)
Harmonic response	Code: PolyDyna
Local stress constraints with an augmented-Lagrangian optimiser	Code: PolyStress (Giraldo-Londoño & Paulino 2021)
Geometry projection (bars with round ends)	Code: GGP-Matlab, GPTO, pygpto (Norato et al.)
Reaction–diffusion level set	Code: 88-line code of Otomori et al. 2015
Hamilton–Jacobi level set; topological derivative	Code: Challis 2010; Novotny et al. 2023
Phase field	Paper: Bourdin & Chambolle 2003; Takezawa et al. 2010
Explicit length-scale constraints, machining accessibility, projection-based self-support	Paper: Zhou et al. 2015; Langelaar 2019; Gaynor & Guest 2016
Multi-resolution design and analysis meshes	Paper: Nguyen et al. 2010
Geometrically non-linear elasticity	Code: 137-line BESO (Han et al. 2021)
Neural reparameterisation (TOuNN)	Code: TOuNN (needs PyTorch or JAX, which may not install on Python 3.14)

What	Source
SINH and Hashin–Shtrikman material laws	Paper: Bruns 2005; Bendsøe & Sigmund 1999 (the material law has to see the volume and Poisson's ratio)

Large: the framework has to change first

What	Unlocks
Several fields per element	DMO, fibre orientation, true multi-material
Unstructured meshes	PolyTop/PolyMesher, VEM, FEniTop, CAD import
Beam and truss elements	ground structures (GRAND), Michell layouts
Multiscale, IGA, CutFEM	de-homogenisation (deHomTop808), IgaTop, GridapTopOpt
Fluids, transient, acoustics, photonics	topflow, TOFLUX, ceviche, Meep
Learned methods	TopoDiff, TopologyGAN, dl4to and the SELTO datasets

Blocked for now

What	Why
IPOPT (`cyipopt`)	no Windows build for Python 3.14; would need conda or an older Python
ParOpt	not on PyPI
TOFLUX	the repository states no licence, so it cannot be reused
MATLAB bridge (run the originals as whole methods)	the MATLAB Engine for R2026a supports Python ≤ 3.13; GNU Octave is the free alternative